

أسئلة الفايصل

مادة تصميم برامج الحاسب

صَلِّ عَلَى النَّبِيِّ ﷺ



اختر الإجابة الصحيحة

1. دورة حياة تطوير البرمجيات (SDLC) تبدأ بـ :

(أ) التصميم (ب) البرمجة (ج) تحديد المشكلة (د) الصيانة

2. الهدف من مرحلة تحليل المتطلبات هو :

(أ) كتابة الكود (ب) فهم احتياجات المستخدم

(ج) اختبار البرنامج (د) تشغيل البرنامج

3 الخوارزمية هي:

(أ) برنامج جاهز (ب) خطوات لحل مشكلة

(ج) لغة برمجة (د) جهاز كمبيوتر

4 أي من التالي ليس طريقة لتمثيل الخوارزمية؟

(أ) Pseudo Code (ب) Flowchart

(ج) لغة الآلة (د) اللغة الطبيعية

5 المخططات الانسيابية تستخدم

(أ) كلمات فقط (ب) رموز وأشكال (ج) أرقام فقط (د) أصوات

6 الشكل المستخدم لبدء وإنهاء الخوارزمية هو:

(أ) مستطيل (ب) معين (ج) بيضاوي (د) متوازي الأضلاع

7. الشكل المستخدم لاتخاذ القرار هو:

(أ) مستطيل (ب) معين (ج) بيضاوي (د) سهم

8 المترجم (Compiler) وظيفته

(أ) تشغيل البرنامج (ب) كتابة الكود
(ج) ترجمة البرنامج (د) حذف الأخطاء

9 لغة الآلة تتكون من:

(أ) حروف (ب) رموز (ج) أرقام ثنائية (د) كلمات

10. المتغير هو:

(أ) برنامج (ب) مكان لتخزين البيانات (ج) دالة (د) شرط

11. في بايثون تعريف المتغير يكون

(أ) لازم نوعه (ب) بدون تحديد نوع
(ج) باستخدام int فقط (د) باستخدام float فقط

12. أي من التالي يمثل عدد عشري؟

(أ) 10 / (ب) "10" (ج) 10.5 (د) True

13. النصوص في بايثون تكتب بين

(أ) أقواس (ب) علامات تنصيص (ج) أرقام (د) رموز

14. الدالة المستخدمة لعرض البيانات هي:

(أ) input (ب) print (ج) type (د) len()

15. الدالة input تستخدم لـ :

(أ) الطباعة (ب) إدخال البيانات (ج) الحساب (د) المقارنة

16. ناتج $3 > 5$ هو

(أ) True (ب) False (ج) Error (د) 0

17. الرمز `==` يعني

(أ) أكبر من (ب) أصغر من (ج) يساوي (د) لا يساوي

18. تعطي `range(2,6)`

(أ) 2,3,4,5 (ب) 2,3,4,5,6 (ج) 0,1,2,3,4,5 (د) 2 إلى 6

19 حلقة for تستخدم عندما

(أ) لا نعرف عدد التكرار (ب) نعرف عدد التكرار (ج) لا نحتاج تكرار (د) نطبع فقط

20 حلقة while تعمل عندما

(أ) الشرط خطأ (ب) الشرط صحيح (ج) لا يوجد شرط (د) مرة واحدة فقط

21. الدالة len تستخدم لـ :

(أ) الطباعة (ب) حساب الطول (ج) الإدخال (د) المقارنة

22 ترتيب العمليات الحسابية الصحيح يبدأ بـ :

(أ) الجمع (ب) الطرح (ج) الأقواس (د) القسمة

23 الرمز % في بايثون يستخدم لـ :

(أ) الجمع (ب) القسمة (ج) باقي القسمة (د) الضرب

24. الكلمة continue تستخدم لـ :

(أ) إنهاء البرنامج (ب) إنهاء الحلقة (ج) تخطي تكرار (د) بدء الحلقة

25 الكلمة break تستخدم لـ :

(أ) تخطي تكرار (ب) إنهاء الحلقة (ج) الطباعة (د) الإدخال

26 تعطي range(5)

(أ) 1 إلى 5 (ب) 0 إلى 4 (ج) 0 إلى 5 (د) 1 إلى 4

27. في بايثون يبدأ فهرس النص من:

1 (أ) 0 (ب) 2 (ج) 1- (د)

28. يمكن دمج النصوص باستخدام

(أ) - (ب) * (ج) + (د) /

29. المتغيرات في بايثون

(أ) حساسة لحالة الأحرف (ب) غير حساسة

(ج) أرقام فقط (د) نصوص فقط

30. لا يمكن أن يبدأ اسم المتغير بـ:

(أ) حرف (ب) _ (ج) رقم (د) كلمة

31. المرحلة التي يتم فيها تثبيت البرنامج على بيئة المستخدم والتأكد من أنه يعمل بشكل صحيح في الواقع تسمى:

(أ) التصميم (ب) البرمجة (ج) التنفيذ والنشر (د) الاختبار

32. كتابة خطوات الحل بشكل منظم يشبه البرمجة بدون التقيد بلغة معينة، يُعرف بـ:

(أ) المخططات الانسيابية (ب) اللغات الطبيعية

(ج) الكود الزائف (Pseudo code) (د) لغة الآلة

33. أي مما يلي يُعد من خصائص المفسر (Interpreter)?

(أ) يترجم البرنامج كاملاً دفعة واحدة

(ب) سريع جداً في التنفيذ

(ج) يكتشف الأخطاء بعد تحليل البرنامج بالكامل

(د) يحلل البرنامج سطراً بسطر ويسهل تتبع الأخطاء

34. أي من لغات البرمجة التالية تعتمد على المترجم (Compiler) بدلاً من المفسر؟

(أ) Python (ب) PHP (ج) C++ (د) Ruby

35. الحد الأقصى لطول اسم المتغير في بايثون هو:

(أ) 10 أحرف (ب) 31 حرفاً أو رقماً

(ج) 50 حرفاً (د) لا يوجد حد أقصى

36. أي من أسماء المتغيرات التالية يُعد صحيحاً ومقبولاً في لغة بايثون؟

(أ) ndPlace2 (ب) full name (ج) temp_ (د) \$total

37. النوع المستخدم لتخزين الأعداد المركبة (التي يحتاجها المهندسون) في بايثون هو:

(أ) int (ب) float (ج) Complex (د) bool

38. في لغة بايثون، أي من العمليات الحسابية التالية لها الأولوية القصوى (تُنفذ أولاً)؟

(أ) الأقواس () (ب) الأسس ** (ج) الضرب والقسمة (د) الجمع والطرح

39. ما هو ناتج تنفيذ جملة الطباعة التالية: `print(3*5+7)` ؟

(أ) 36 (ب) 22 (ج) 50 (د) 15

40. ما هو ناتج تنفيذ العملية الحسابية التالية: `print(8+2)*5-12/4` ؟

(أ) 47 (ب) 22 (ج) 50 (د) 38

41. تُستخدم الكلمة المحجوزة `elif` في جمل الشرط بـ بايثون لـ:

(أ) إنهاء البرنامج (ب) تنفيذ كود إذا لم يتحقق أي شرط سابق

(ج) اختبار شرط إضافي جديد إذا كان الشرط الأول خاطئاً

(د) تعريف متغير جديد

42. أي من الحلقات التكرارية التالية هي الأنسب عندما نكون بانتظار حدث معين

ولا نعرف عدد التكرارات مسبقاً؟

(أ) `for` (ب) `while` (ج) `If` (د) `range`

43. ما هي الأرقام التي ستنتج من الدالة `range(10, 2, -2)` ؟

(أ) 4, 6, 8, 10 (ب) 2, 4, 6, 8, 10

(ج) 10, 8, 6, 4, 2 (د) 2, 4, 6, 8

44. مجموعة مرتبة من العناصر "قابلة للتغيير والتعديل"، وتُكتب بين أقواس مربعة [] هي:

- (أ) النصوص (String) (ب) الصفوف (Tuple)
(ج) القوائم (List) (د) المتغيرات العادية

45. مجموعة مرتبة من العناصر "غير قابلة للتغيير"، وتُكتب بين أقواس دائرية () هي:

- (أ) النصوص (String) (ب) الصفوف (Tuple)
(ج) القوائم (List) (د) القواميس (Dictionary)

46. إذا كان لدينا القائمة `my_list = [5, 15, 25, 53, 45, 55]`، فما هو ناتج التقطيع `[2:5]my_list` ؟

- (أ) [15, 25, 35] (ب) [25, 35, 45, 50]
(ج) [25, 35, 45] (د) [5, 15, 25]

47. إذا كان لدينا القائمة `my_list = [5, 15, 25, 35, 45, 55]`، فما هو ناتج `[4:]my_list` ؟

- (أ) [45, 55] (ب) (45, 55)
(ج) [55, 45, 35, 25] (د) [55, 45, 35]

48. أي من العمليات التالية "خاطئة" ولا يمكن تنفيذها على الصفوف (Tuple)؟

(أ) my_tuple[0] (ب) len(my_tuple)

(ج) print(my_tuple) (د) my_tuple.append(15)

49. في حلقة التكرار، الدالة التي تقوم بـ "إيقاف الحلقة بالكامل والخروج منها فوراً" هي:

(أ) continue (ب) break (ج) pass (د) elif

50. إذا أردت استخراج عناصر القائمة بدءاً من الفهرس 3 وحتى النهاية، تكتب:

(أ) my_list[3:] (ب) my_list[:3]

(ج) my_list[3:0] (د) my_list[0:3]

51. أي من التالي يُعد الوصف الأدق للمجموعات (Sets) في بايثون؟

(أ) عناصر مرتبة يمكن الوصول إليها بواسطة الـ Index.

(ب) مصفوفة لتخزين بيانات من نفس النوع فقط.

(ج) مجموعة غير مرتبة من العناصر الفريدة (لا تقبل التكرار).

(د) مجموعة مرتبة تقبل تكرار القيم.

52. لتعريف مجموعة (Set) جديدة في بايثون، نستخدم:

(أ) الأقواس المربعة [] (ب) الأقواس المعكوفة { } أو الدالة set()

(ج) الأقواس العادية () (د) الكلمة المحجوزة list

53. لإضافة عنصر جديد للمجموعة (Set)، نستخدم الدالة:

(أ) append (ب) insert (ج) add (د) union

54. العملية $A - B$ في المجموعات تعني:

- (أ) دمج كل العناصر في المجموعتين.
- (ب) استخراج العناصر المشتركة بين المجموعتين.
- (ج) استخراج العناصر الموجودة في B وغير موجودة في A.
- (د) استخراج العناصر الموجودة في A وغير موجودة في B.

55. الرمز | في التعامل مع المجموعات (Sets) يُستخدم لـ:

- (أ) إيجاد التقاطع (Intersection).
- (ب) إيجاد الدمج أو الاتحاد (Union).
- (ج) حساب الفرق (Difference).
- (د) حذف جميع العناصر.

56. ما هي الكلمة المحجوزة التي تُستخدم لتعريف دالة (Function) جديدة في بايثون؟

(أ) fun (ب) function (ج) def (د) define

57. تُستخدم الأقواس () التي تلي اسم الدالة عند تعريفها في:

- (أ) كتابة جسم الدالة.
- (ب) تمرير القيم أو الوسائط (Parameters).
- (ج) إرجاع القيم للخارج.
- (د) إيقاف تنفيذ الدالة.

58. لإرسال أو إرجاع قيمة من الدالة إلى المكان الذي استدعيت منه، نستخدم الكلمة المحجوزة:

(أ) export (ب) send (ج) return (د) output

59. الدالة مجهولة الاسم التي تتألف من سطر واحد فقط في بايثون تُعرف باستخدام الكلمة:

(أ) lambda (ب) inline (ج) def (د) anonymous

60. الكود `print(set([1, 1, 2, 2, 3]))` سيقوم بطباعة:

(أ) [1, 1, 2, 2, 3] (ب) {1, 2, 3} (ج) () (د) Error

61. لتضمين (استدعاء) موديول أو مكتبة في ملف بايثون، نستخدم الكلمة المحجوزة:

(أ) include (ب) export (ج) import (د) require

62. ماذا يعني الكود `from math import sqrt, pi` ؟

(أ) استدعاء الموديول `math` كاملاً بكل دواله.

(ب) استدعاء الدالة `sqrt` والمتغير `pi` فقط من الموديول `math`.

(ج) إعطاء الموديول `math` اسماً مختصراً.

(د) استدعاء الموديول وتغيير اسمه إلى `sqrt`.

63. "مخطط" أو "قالب" يتم منه إنشاء الكائنات في البرمجة الشيئية (OOP) يُسمى:

- (أ) الكائن (Object). (ب) الدالة (Method).
(ج) الكلاس (Class). (د) التغليف (Encapsulation).

64. الأفعال أو المهام التي يمكن للكائن القيام بها تُسمى:

- (أ) الخصائص (Attributes). (ب) الكلاسات (Classes).
(ج) المتغيرات (Variables). (د) الدوال (Methods).

65. الطريقة الصحيحة لإنشاء كائن جديد من كلاس يُسمى Student هي:

- (أ) `S1 = Student()` (ب) `S1 == Student`
(ج) `Student S1 = new Student()` (د) `class Student = S1`

66. الدالة الجاهزة التي تُنفذ تلقائياً بمجرد إنشاء كائن جديد وتُستخدم لإعطاء قيم أولية لخصائصه هي:

- (أ) `__start__()` (ب) `__init__()`
(ج) `__main__()` (د) `__constructor__()`

67. الكلمة التي يجب أن تكون أول Parameter في أي دالة يتم تعريفها داخل الكلاس للإشارة إلى الكائن الحالي هي:

- (أ) `this` (ب) `object` (ج) `self` (د) `me`

68. مفهوم حماية بيانات الكلاس والتحكم في الوصول إليها لمنع التعديل الخاطئ يُسمى:

- (أ) التغليف (Encapsulation). (ب) الوراثة (Inheritance).
(ج) التجريد (Abstraction). (د) تعدد الأشكال (Polymorphism).

69. المتغير الذي يبدأ بشرطتين سفليتين (مثل password__) يُعتبر من النوع:

- (أ) Public (ب) Protected (ج) Private (د) Global

70. المتغير الذي يُستخدم داخل الكلاس والكلاسات الوريثة فقط (ويبدأ بشرطة سفلية واحدة name__) يُعتبر من النوع:

- (أ) Public (ب) Protected (ج) Private (د) Local

71. لكتابة كلاس جديد (Child) يرث من كلاس أساسي (Parent) في بايثون، نستخدم الصيغة:

(أ) :class Child inherits Parent

(ب) :class Child(Parent)

(ج) :class Child -> Parent

(د) :class Parent(Child)

72. إخفاء التفاصيل المعقدة للكود وإظهار الوظائف الأساسية فقط للمستخدم يُعرف بـ:

- أ) الوراثة (Inheritance).
 ب) تعدد الأشكال (Polymorphism).
 ج) التغليف (Encapsulation).
 د) التجريد (Abstraction).

73. قدرة نفس الدالة على تنفيذ سلوكيات مختلفة بناءً على الكائن الذي يستدعيها تُسمى:

- أ) التجريد (Abstraction).
 ب) التغليف (Encapsulation).
 ج) تعدد الأشكال (Polymorphism).
 د) الوراثة (Inheritance).

74. الدالة المجردة (Abstract Method) تُعرف عادةً بدون كتابة محتوى لها، ولتجنب ظهور خطأ نستخدم بداخلها الكلمة المحجوزة:

- أ) stop ب) break ج) continue د) pass

75. عند الوصول إلى أمر return داخل أي دالة، فإن بايثون:

- أ) تعود لبداية الدالة وتكررها.
 ب) تتوقف عن تنفيذ أي كود آخر داخل الدالة وتخرج منها.
 ج) تتجاهله وتستمر في تنفيذ باقي السطور.
 د) تقوم بطباعة القيمة على الشاشة فقط دون إرجاعها.

1. دورة حياة تطوير البرمجيات هي مجموعة من المراحل المنظمة من الفكرة حتى التسليم والصيانة. (✓)
2. مرحلة جمع وتحليل المتطلبات تهدف لفهم ما يحتاجه المستخدم وتحديد الوظائف المطلوبة. (✓)
3. مرحلة التصميم تعني كتابة الكود الفعلي للبرنامج. (✗) (البرمجة)
4. الصيانة والدعم تشمل إصلاح الأخطاء بعد نشر البرنامج. (✓)
5. في مرحلة تحديد المشكلة، يقوم المبرمج بتحديد الهدف من البرنامج ونوع المخرجات. (✓)
6. الخوارزمية هي عبارة عن الخطوات اللازمة لحل مسألة ما. (✓)
7. استخدام اللغات الطبيعية هو طريقة مباشرة للتعبير عن الخوارزمية. (✓)
8. من عيوب استخدام اللغات الطبيعية طولها أحياناً وعدم الدقة. (✓)
9. الكود الزائف (Pseudo Code) هو أحد الطرق البديلة للتعبير عن الخوارزمية. (✓)
10. المخططات الانسيابية (Flow Charts) هي تمثيل رمزي للخوارزمية. (✓)
11. في أي برنامج يجب ترجمة البرنامج من لغة المستوى العالي إلى لغة الآلة. (✓)
12. المدخلات هي النتائج المطلوبة من البرنامج. (✗) (التصحيح: المخرجات)
13. خرائط سير العمليات تستخدم رموزاً اصطلاحية لتوضيح خطوات الحل. (✓)
14. الشكل البيضاوي في خرائط التدفق يستخدم لتحديد البداية والنهاية. (✓)
15. شكل المستطيل يستخدم للتعبير عن الإدخال والإخراج. (✗) (التصحيح: متوازي الأضلاع)
16. شكل متوازي الأضلاع يستخدم للتعبير عن الإدخال والإخراج. (✓)
17. شكل المستطيل يستخدم للتعبير عن العمليات. (✓)
18. شكل المعين يستخدم للتعبير عن العمليات المنطقية واتخاذ القرار. (✓)
19. الأسهم في خرائط التدفق تحدد اتجاه سير البرنامج. (✓)
20. خرائط التتابع البسيط هي أحد أنواع خرائط التدفق. (✓)
21. الخرائط ذات الفروع تستخير (تستخدم) عندما يكون هناك سؤال يحتاج لإجابة نعم أو لا. (✓)

22. الدقة هي إحدى مزايا استخدام اللغات الطبيعية في الخوارزميات. (☒) (التصحيح: اللغات الطبيعية يعيبها عدم الدقة والغموض أحياناً)
23. المترجم (Compiler) يقوم بتحويل لغة المستوى العالي إلى لغة الآلة. (☒)
24. يمكن أن تحتوي خريطة التدفق على أكثر من إجابتين للسؤال المنطقي. (☒) (التصحيح: إجابتين فقط عادة: نعم أو لا)
25. خرائط الدورانات تستخدم لإعادة عملية أو مجموعة عمليات عدة مرات. (☒)
26. لغة الآلة هي اللغة التي يفهمها الحاسب مباشرة. (☒)
27. الطبع "المجموع" تعتبر خطوة معالجة بيانات. (☒) (التصحيح: الطبع يعتبر عملية إخراج (Output))
28. خرائط التدفق تعتبر أسهل لأنها لا تحتاج للتعبير باللغة كثيراً. (☒)
29. الخطوة الأخيرة في خريطة التدفق هي "البداية". (☒) (التصحيح: "النهاية")
30. الكمبيوتر يفهم لغة البشر العادية بشكل مباشر. (☒) (التصحيح: الكمبيوتر يفهم لغة الآلة 0 و 1 فقط)
31. لغة الآلة هي لغة منخفضة المستوى تتكون من أرقام ثنائية. (☒)
32. لغات البرمجة تعمل كوسيط لتقريب الفجوة بين البشر ولغة الآلة. (☒)
33. المتغير هو مكان في ذاكرة الكمبيوتر لتخزين البيانات. (☒)
34. عند تعريف متغير في بايثون، يجب تحديد نوعه مثل (int) بشكل يدوي قبل الاسم. (☒) (التصحيح: تحدد نوع المتغير تلقائياً)
35. الاسم age و الاسم AGE يعتبران متغيراً واحداً في بايثون. (☒) (التصحيح: بايثون لغة حساسة لحالة الأحرف)
36. لا يمكن أن يبدأ اسم المتغير برقم. (☒)
37. النوع int يستخدم لتخزين الأعداد الصحيحة مثل 10 و 50. (☒)
38. النوع float يستخدم لتخزين الأعداد التي تحتوي على فاصلة عشرية. (☒)
39. السلاسل النصية string يجب أن توضع بين علامات تنصيص. (☒)

40. النوع bool يأخذ قيمتين فقط هما True أو False. (☒)
41. الدالة type () تستخدم لمعرفة نوع المتغير. (☒)
42. الدالة input () تستخدم لعرض النتائج على الشاشة. (☒) (التصحيح: تستخدم لإدخال واستقبال البيانات)
43. القيمة التي ترجع من الدالة input () تكون دائماً من نوع نص. (☒)
44. الرمز % في بايثون يستخدم لحساب النسبة المئوية. (☒) (التصحيح: الرمز % يستخدم لحساب "باقي القسمة")
45. الضرب والقسمة لهما أولوية في التنفيذ قبل الجمع والطرح. (☒)
46. الأقواس لها أعلى أولوية في العمليات الحسابية. (☒)
47. العملية 2 ** 3 في بايثون تعني 2 مضروبة في 3. (☒) (التصحيح: الرمز ** يعني "الأس"، أي 2 أس 3 = 8)
48. الدالة print () تُستخدم لطباعة المخرجات للمستخدم. (☒)
49. المتغير من النوع bool يمكن أن يحمل القيمة 8. (☒) (التصحيح: متغير bool يحمل True أو False فقط)
50. بايثون لغة تدعم العمليات الحسابية الأساسية مثل الجمع والطرح والضرب والقسمة. (☒)
51. الدالة input () تستخدم لاستقبال البيانات من المستخدم أثناء تشغيل البرنامج. (☒)
52. ترجع الدالة input () دائماً قيمة من النوع الرقمي int. (☒) (التصحيح: ترجع قيمة نصية String دائماً)
53. لتحويل النص المدخل من المستخدم إلى رقم عشري نستخدم الدالة float(). (☒)
54. الرمز == يُستخدم لمقارنة ما إذا كانت قيمتان متساويتين. (☒)
55. الرمز = يعني "يساوي" في لغة بايثون. (☒) (التصحيح: الرمز = يستخدم لتخزين القيمة)
56. التعبير (5>3) يعطي نتيجة True. (☒)
57. التعبير (10<2) يعطي نتيجة True. (☒) (التصحيح: يعطي نتيجة False لأن 2 ليست أكبر من 10)

58. تستخدم الجملة الشرطية if لاتخاذ القرارات بناءً على شرط معين. (☒)
59. في جملة if البسيطة، يتم تنفيذ الكود التابع لها فقط إذا كان الشرط صحيحة (True). (☒)
60. الكلمة المحجوزة else تُستخدم لتنفيذ كود معين في حال كان شرط if خاطئاً (False). (☒)
61. العامل المنطقي and يعطي True إذا كان أحد الشرطين فقط صحيحاً. (☒) (التصحيح: and يعطي True فقط إذا كان "كلا" الشرطين صحيحين)
62. العامل المنطقي or يعطي True إذا تحقق شرط واحد على الأقل من الشرطين. (☒)
63. العامل not يعكس الحالة المنطقية، فإذا كانت القيمة True يحولها إلى False. (☒)
64. في بايثون، يجب وضع نقطتين رئيسيتين : بعد جملة if وجملة else. (☒)
65. المسافة البادئة (Indentation) قبل الكود التابع لـ if اختيارية ولا تؤثر على البرنامج. (☒) (التصحيح: تأثر على هيكل البرنامج)
66. يمكن استخدام int(input()) لقراءة عمر المستخدم كعدد صحيح مباشرة. (☒)
67. المقارنة (b<=a) تعني أن a أكبر من أو تساوي b. (☒)
68. المقارنة (b>=a) تعني أن a أصغر من b فقط. (☒) (التصحيح: اصغر من او تساوي b)
69. إذا كان age = 12 فإن الشرط age>18 سيعطي False. (☒)
70. نستخدم float() إذا كنا نتوقع من المستخدم إدخال طوله بالمتر (مثلاً 1.75). (☒)
71. العامل (5<3)not يعطي نتيجة True. (☒)
72. جملة if يمكن أن تعمل بمفردها دون الحاجة لوجود else. (☒)
73. range(1,5) تنتج الأرقام 1، 2، 3، 4، 5. (☒) (التصحيح: الدالة تنتج الأرقام 1, 2, 3, 4 فقط، الرقم الأخير غير مشمول)
74. تستخدم حلقة for عندما نعرف عدد التكرارات مسبقاً. (☒)
75. في لغة بايثون، يبدأ ترتيب الحروف في النصوص من الرقم 1. (☒) (التصحيح: من الرقم صفر 0)
76. يمكن دمج النصوص باستخدام علامة الجمع (+). (☒)
77. الدالة len() تستخدم لمعرفة عدد أحرف النص. (☒)

78. حلقة while تستخدم لتكرار الأوامر طالما الشرط صحيح. (☒)
79. الكلمة continue تنهي الحلقة التكرارية بالكامل. (☒) (التصحيح: continue تتخطى الدورة الحالية فقط وتكمل الحلقة، بينما break هي من تنهي الحلقة بالكامل)
80. range(5) تبدأ العد من 0 وتنتهي عند 4. (☒)
81. يمكن كتابة النصوص في بايثون باستخدام علامات تنصيص مفردة أو مزدوجة. (☒)
82. المتغير i داخل الحلقة التكرارية هو مجرد اسم ويمكن تغييره لأي اسم آخر. (☒)
83. إذا كان s = "برمجة" فإن s[0] يساوي حرف "ب". (☒)
84. الكود i+=1 هو اختصار لـ i = i + 1. (☒)
85. إذا أردنا تقليل العداد، نستخدم i-=1. (☒)
86. الدالة range يمكنها إنشاء تسلسل من الأرقام فقط. (☒)
87. النصوص في بايثون هي سلسلة من الحروف أو الرموز. (☒)
88. range(1, 5) تعطي الأرقام (1, 2, 3, 4). (☒)
89. الجملة print(s[10]) ستنتج دائماً بغض النظر عن طول النص. (☒) (التصحيح: ستعطي خطأ IndexError إذا كان طول النص أقل من أو يساوي 10)
90. continue تُستخدم لتخطي تكرار معين داخل الحلقة. (☒)
91. الحلقة التكرارية تستخدم لجعل الكود يعاد تنفيذه لعدد محدد أو شرط معين. (☒)
92. لا يمكن دمج نص مع رقم مباشرة باستخدام + دون تحويل الرقم لنص. (☒)
93. range(start, stop, step) هو الترتيب الصحيح لمعاملات الدالة. (☒)
94. len("Hello") تعطي النتيجة 5. (☒)
95. النصوص تعتبر نوعاً من أنواع البيانات في بايثون. (☒)
96. مرحلة الاختبار في تطوير البرمجيات تهدف إلى التأكد من أن البرنامج يعمل بدون أخطاء وتحقيق المتطلبات. (☒)
97. مرحلة التنفيذ والنشر تعني تثبيت البرنامج على بيئة المستخدم للتأكد من عمله في الواقع. (☒)

98. المفسر (Interpreter) يقوم بترجمة البرنامج كاملاً دفعة واحدة. (☒) (التصويب: يحلل ويترجم البرنامج سطرًا بسطر)
99. المفسر (Interpreter) أسرع في التنفيذ من المترجم (Compiler). (☒) (التصويب: المفسر أبطأ في التنفيذ)
100. لغة بايثون (Python) تعتبر من اللغات التي تعتمد على المترجم (Compiler). (☒) (التصويب: تعتمد على المفسر Interpreter)
101. لغات مثل C و C++ تستخدم المترجم (Compiler). (☒)
102. الحد الأقصى لطول اسم المتغير في بايثون هو 50 حرفاً. (☒) (التصويب: من 1 إلى 31 حرفاً أو رقماً)
103. يُسمح باستخدام جميع الرموز الخاصة مثل @ و # في تسمية المتغيرات في بايثون. (☒) (التصويب: يُسمح فقط باستخدام الشرطة السفلية _)
104. يستخدم النوع (Complex) لتخزين الأعداد المركبة ويكتب الحرف j أو J بعد العدد. (☒)
105. القوائم (Lists) هي مجموعة مرتبة من العناصر غير قابلة للتغيير. (☒) (التصويب: القوائم قابلة للتغيير والتعديل، بينما الصفوف Tuples هي الثابتة)
106. تُكتب الصفوف (Tuples) بين أقواس مربعة []. (☒) (التصويب: تُكتب بين أقواس دائرية ()، بينما القوائم تستخدم [])
107. يمكن إضافة عناصر جديدة للقائمة (List) باستخدام الدالة append(). (☒)
108. يمكن تعديل عناصر الصفوف (Tuples) بعد تعريفها. (☒) (التصويب: لا يمكن التغيير، فهي للقراءة فقط وتخزين البيانات الثابتة)
109. يمكن أن تحتوي القوائم (Lists) والصفوف (Tuples) على أنواع بيانات مختلطة (أرقام ونصوص معاً). (☒)
110. إذا كانت القائمة my_list = [5, 15, 25, 35, 45, 55]، فإن التقطيع my_list[4:] يعطي العناصر من البداية وحتى الفهرس 4 (غير مشمول). (☒)
111. الدالة break تنهي الحلقة التكرارية نهائياً وتخرج منها مهما تبقى من عناصر. (☒)
112. الرمز // في بايثون يستخدم للقسمة العادية التي ترجع رقماً عشرياً. (☒) (التصويب: يستخدم للقسمة الصحيحة بدون كسور)

113. في لغة بايثون، عند تساوي العمليات الحسابية في نفس المستوى (مثل الضرب والقسمة)، يتم التنفيذ من اليسار إلى اليمين. (☒)
114. المتغير ndPlace2 يعتبر اسماً صحيحاً ومقبولاً في بايثون. (☒) (التصويب: اسم خاطئ لأنه يبدأ برقم)
115. الكود my_tuple.pop() يعتبر كوداً صحيحاً لحذف عنصر من الصفوف. (☒) (التصويب: خاطئ لأن الصفوف Tuples لا تدعم الحذف أو التعديل)
116. في خريطة تدفق "حجز موعد طبيب"، تعتبر خطوة "اطبع رسالة تأكيد الحجز" من المخرجات. (☒)
117. الدالة extend() تستخدم لإضافة عناصر إلى القوائم (Lists). (☒)
118. النوع set يمثل مصفوفة يمكنها تخزين قيم من مختلف الأنواع في وقت واحد. (☒) - التصويب: الـ set هي "مجموعة" غير مرتبة
119. المجموعات (sets) في بايثون هي عناصر مرتبة ويمكن التعامل معها بواسطة الـ index. (☒) - التصويب: المجموعات غير مرتبة ولا يمكن الوصول لعناصرها باستخدام الـ index.
120. لا يمكن للمجموعة set أن تحتوي على قيم مكررة. (☒)
121. لتعريف set نستخدم الرمز [] وبداخله نمرر القيم. (☒) - التصويب: نستخدم الرمز { } أو الدالة set()، بينما [] تُستخدم للقوائم (Lists).
122. نستخدم الدالة add لإضافة عنصر جديد للمجموعة set. (☒)
123. الدالة remove تُستخدم لحذف الـ set من الذاكرة نهائياً. (☒) - التصويب: تُستخدم لحذف "عنصر محدد"
124. الرمز | أو الدالة union() تستخدم لدمج كل العناصر في مجموعتين بدون تكرار. (☒)
125. الرمز & يستخدم لاستخراج العناصر المشتركة فقط بين مجموعتين. (☒)
126. العملية A - B تعني استخراج العناصر الموجودة في A وغير موجودة في X. (☒) - التصويب: العكس
127. الدالة هي عبارة عن كود برمجي جاهز يتنفذ فقط عند استدعائه. (☒)

128. الغرض من الدالة هو كتابة الكود لمرة واحدة واستخدامه عدة مرات. (✓)
129. نستخدم الكلمة المحجوزة "fun" لتعريف دالة جديدة في بايثون. (✗) - التصويب: **نستخدم الكلمة المحجوزة "def".**
130. الأقواس () التي تلي اسم الدالة تستخدم لتمرير القيم أو الوسائط (Parameters). (✓)
131. النقطتان الرأسيتان : بعد تعريف الدالة ضروريتان لبدء جسم الدالة. (✓)
132. يتم استدعاء الدالة في بايثون عن طريق كتابة اسمها متبوعاً بالأقواس (). (✓)
133. لا يمكن للدالة في بايثون أن ترجع قيمة للمكان الذي استدعيت منه. (✗) - التصويب: **يمكن للدالة إرجاع قيمة باستخدام الكلمة المحجوزة return.**
134. الكلمة المحجوزة "return" تستخدم لإرسال قيمة من الدالة إلى الخارج. (✓)
135. بمجرد تنفيذ أمر return ، تتوقف الدالة عن تنفيذ أي كود بعدها. (✓)
136. في بايثون، يجب دائماً تعريف الدالة قبل استدعائها في الكود. (✓)
137. أسلوب Lambda يُستخدم لتعريف دوال مجهولة الاسم تتألف من سطر واحد فقط. (✓)
138. نستخدم الرمز { } لتعريف القواميس (Dictionaries) أيضاً في بايثون. (✓)
139. الكود `print(set([1, 1, 2, 2, 3]))` سينتج عنه المخرجات {1,2,3}. (✓)
140. الدالة التي تقوم بالتحقق إذا كان الرقم زوجي أم فردي تستخدم معامل % (باقي القسمة). (✓)
141. الموديول (Module) هو عبارة عن ملف بايثون عادي امتداده .py. (✓)
142. يمكن أن يحتوي الموديول على متغيرات ودوال وكلاسات فقط. (✗) - التصويب: **يمكن أن يحتوي أيضاً على أكواد تنفيذية قابلة للتشغيل المباشر.**
143. نستخدم الكلمة المحجوزة export لتضمين محتوى موديول في موديول آخر. (✗) - التصويب: **نستخدم الكلمة المحجوزة import.**
144. عند استخدام `from math import *` فإن النجمة تعني تضمين كل شيء موجود في الموديول. (✓)
145. يمكن إعطاء اسم مختصر للموديول باستخدام الكلمة المحجوزة `as` مثل `import math as m`. (✓)

146. الكود `from math import sqrt, pi` يقوم باستدعاء الموديول كاملاً بكل دواله. (X)
147. البرمجة كائنية التوجه (OOP) هي أسلوب لبرمجة الحاسوب يعتمد على تقسيم البرنامج إلى "كائنات". (✓)
148. الكلاس (Class) يُعتبر بمثابة "مخطط" أو "قالب" يتم منه إنشاء الكائنات. (✓)
149. نستخدم الكلمة المحجوزة `class` متبوعة باسم الكلاس لتعريف كلاس جديد. (✓)
150. "الكائن (Object)" هو نسخة ملموسة يتم إنشاؤها من الكلاس. (✓)
151. الخصائص (Attributes) هي البيانات أو المتغيرات التي تنتمي للكائن. (✓)
152. الأفعال أو المهام التي يمكن للكائن القيام بها تسمى "Methods". (✓)
153. لإنشاء كائن جديد من كلاس `student` نكتب `S1 = Student()`. (✓)
154. نستخدم النقطة `.` للوصول إلى خصائص أو دوال الكائن، مثل `name`. (✓)
155. جميع الكلاسات في بايثون تملك دالة جاهزة تسمى `__init__()`. (✓)
156. دالة `__init__()` تنفذ فقط عندما نقوم باستدعائها يدوياً في الكود. (X) - التصويب: هذه الدالة (Constructor) تُنفذ تلقائياً بمجرد إنشاء كائن جديد من الكلاس.
157. نستخدم دالة `__init__()` لإعطاء قيم أولية لخصائص الكائن عند إنشائه مباشرة. (✓)
158. الكلمة المحجوزة `self` تستخدم للإشارة إلى الكائن الحالي الذي يتم التعامل معه داخل الكلاس. (✓)
159. يجب أن يكون `self` هو أول "Parameter" في أي دالة يتم تعريفها داخل الكلاس. (✓)
160. لا يمكن تمرير قيم لدالة `__init__()` من خارج الكلاس. (X) - التصويب: يمكن تمرير قيم لها عبر المعاملات (Parameters) عند إنشاء الكائن.
161. في بايثون، يمكن للكلاس الواحد أن يحتوي على أكثر من دالة `__init__()`. (X)
162. الكود `name.greet()` يقوم باستدعاء الدالة `greet` وتمرير اسم الطالب لها. (X)
163. مبادئ البرمجة الشيئية (OOP) الأساسية هي أربعة: التغليف، الوراثة، التجريد، وتعدد الأشكال. (✓)
164. التغليف (Encapsulation) يعني حماية بيانات الكلاس والتحكم في الوصول إليها. (✓)

165. يساعد التغليف على حماية البيانات من التعديل الخاطئ وزيادة الأمان. (✓)
166. المتغير من النوع Public يمكن الوصول إليه من أي مكان داخل أو خارج الكلاس. (✓)
167. المتغير الذي يبدأ بشرطة سفلية واحدة مثل name_ يعتبر من النوع Private. (✗) - التصويب: المتغير الذي يبدأ بشرطة سفلية واحدة يعتبر Protected، أما Private فيبدأ بشرطتين سفليتين. ____
168. المتغير من النوع Protected يُستخدم داخل الكلاس والكلاسات الوريثة فقط. (✓)
169. المتغير الذي يبدأ بشرطتين سفليتين مثل password__ هو متغير Private لا يمكن الوصول إليه مباشرة خارج الكلاس. (✓)
170. الوراثة (Inheritance) تسمح لكلاس جديد بأن يرث الخصائص والدوال من كلاس آخر. (✓)
171. من فوائد الوراثة تقليل تكرار الكود وإعادة استخدامه. (✓)
172. في بايثون، لكتابة كلاس يرث من كلاس آخر نضع اسم الكلاس الأساسي بين أقواس بجانب اسم الكلاس الجديد. (✓)
173. التجريد (Abstraction) هو إخفاء التفاصيل المعقدة وإظهار الوظائف الأساسية فقط للمستخدم. (✓)
174. الهدف من التجريد هو زيادة تعقيد الكود لضمان الحماية. (✗)
175. الدالة المجردة (Abstract Method) هي دالة تُعرف بدون كتابة محتوى لها (تستخدم pass). (✓)
176. تعدد الأشكال (Polymorphism) هو قدرة نفس الدالة على تنفيذ سلوكيات مختلفة بناءً على الكائن الذي يستدعيها. (✓)
177. يسمح تعدد الأشكال باستخدام واجهة واحدة (اسم دالة واحد) لتنفيذ مهام مختلفة. (✓)
178. التغليف والوراثة هما نفس المفهوم في البرمجة الشيئية. (✗) - التصويب: هما مفهومان مختلفان؛ التغليف لحماية البيانات وإخفائها، والوراثة لنقل الخصائص من كلاس لآخر وإعادة استخدام الكود.
179. الوراثة تسهل تنظيم الكلاسات بشكل أفضل وتطوير البرامج. (✓)
180. البرمجة الشيئية تجعل الكود أكثر تنظيماً وقابلية لإعادة الاستخدام. (✓)

أسئلة المقالى الامتحان هىكون 3 أسئلة ممكن سؤال خرائط - مانتج الكود - واكتب برنامج

قبل ما نحل.. يعنى إيه خريطة تدفق (Flowchart)؟

خريطة التدفق هي ببساطة "رسمة" بتوضح خطوات حل أي مشكلة أو برنامج من البداية للنهاية عشان نفهمه قبل ما نكتب الكود. كل شكل هندسي بنرسمه ليه وظيفة معينة، ومينفعش نبدهم ببعض!

تعالوا نتعرف على الأشكال الأساسية ووظيفة كل واحد:

1. الشكل البيضاوي (Oval) :

ده بنستخدمه مرتين بس في الرسمة كلها! مرة في أول خطوة فوق خالص وبنكتب جواه (البداية / Start)، ومرة في آخر خطوة تحت خالص وبنكتب جواه (النهاية / End).

2. متوازي الأضلاع (Parallelogram) :

ده بتاع الـ "دخول والخروج".

بنستخدمه لما نعوز ناخذ معلومة من المستخدم (إدخال / Input).. زي مثلاً "أدخل طول الضلع".

أو لما نعوز نطلع له نتيجة على الشاشة (طباعة / Print).. زي مثلاً "اطبع المساحة".

3. المستطيل (Rectangle) :

ده "المطبخ" أو عقل البرنامج! بنحط جواه أي عملية حسابية أو معادلة،

زي مثلاً (المساحة = الطول × العرض) أو (الناتج = س + ص).

4. المَعِين (Diamond) :

ده بتاع "الأسئلة واتخاذ القرار".

بنحط جواه أي شرط إجابته (أيوة أو لا).. زي مثلاً: هل العمر أكبر من 18؟

وبيطلع من المعين ده سهمين: سهم بنمشي فيه لو الإجابة صح (Yes)، وسهم تاني لو الإجابة غلط (No).

5. خطوط الاتجاه (السهم) :

ودي الأسهم اللي بتربط الأشكال دي كلها ببعضها عشان تعرفنا البرنامج ماشي إزاي من خطوة للتانية.

1- ارسم خريطة التدفق التي تحسب ناتج جمع رقمين مع كتابة خطوات الحل.



المدخلات : رقم 1 و رقم 2

المخرجات : المجموع

الخطوات

1. البداية

2. اقرأ رقم 1 و رقم 2

3. احسب المجموع = رقم 1 + رقم 2

4. اطبع المجموع

5. النهاية

2- ارسم خريطة التدفق لحساب متوسط وحاصل ضرب ثلاثة اعداد



المدخلات : د1 و د2 و د3

المخرجات : المتوسط وحاصل ضرب

الخطوات

1. البداية

2. اقرأ د1 و د2 و د3

3. احسب المتوسط = $(د1+د2+د3)/3$

4. احسب حاصل ضرب = $د1*د2*د3$

5. اطبع المتوسط وحاصل ضرب

6. النهاية

-3

حسب مساحة المستطيل بمعلومية الطول والعرض ، قم بتحليل عناصر المسألة إذا علمت أن مساحة المستطيل = الطول × العرض.

المدخلات : الطول والعرض

المخرجات : المساحة

الخطوات

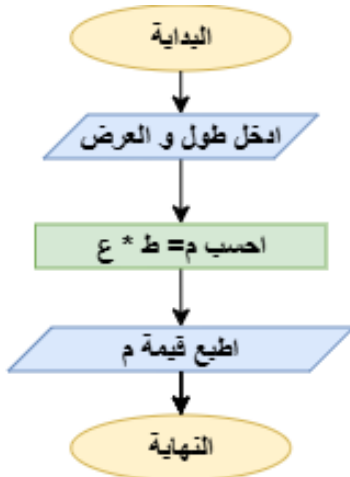
1. البداية

2. أدخل الطول (ط) والعرض (ع).

3. حساب مساحة المستطيل نرسم لها بالرمز م $ط * ع$

4. طباعة قيمة مساحة المستطيل م

5. النهاية



-4

ارسم خريطة التدفق لإدخال رقم ثم طباعة نوع العدد (زوجي أو فردي) ؟

المدخلات : ادخل N

المخرجات : طباعة الرقم (زوجي أو فردي)

الخطوات

1. البداية

2. أدخل الطول (ط) والعرض (ع).

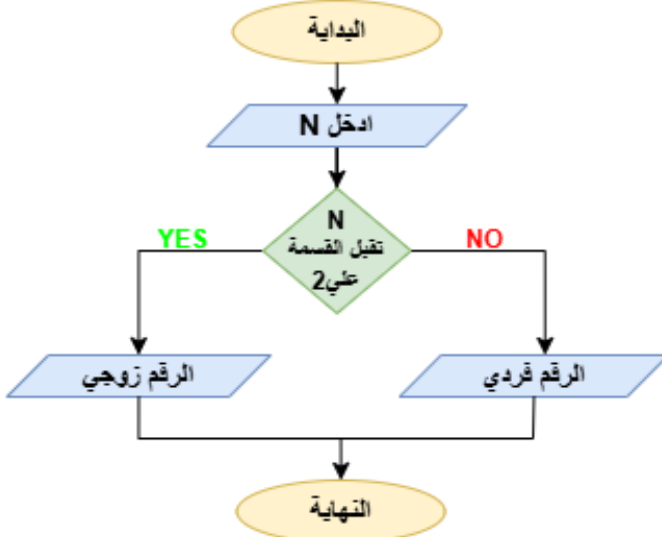
3. إذا كانت N تقبل القسمة علي 2

1-3 اطبع الرقم زوجي

4. غير ذلك:

1-4 اطبع رقم فردي

5. النهاية

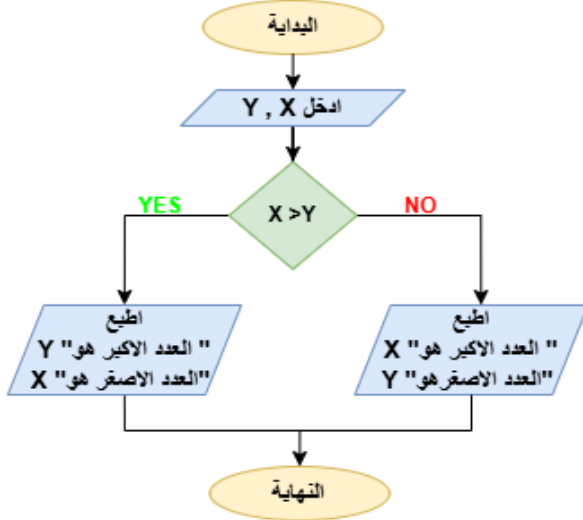


-5

ارسم خريطة التدفق لإدخال عددين ثم طباعة "العدد الأكبر هو ؟" و "العدد الأصغر هو."

المدخلات : أدخل العدد X و العدد Y

المخرجات : طباعة عبارة "العدد الأكبر هو ؟" و "العدد الأصغر هو



1. البداية

2. أدخل X, Y

3. إذا كان XY

1-3 طباعة " العدد الأكبر هو X والعدد الأصغر هو Y "

3 - 2 إذهب للخطوة رقم 6 او تكتب نهاية

4. غير ذلك:

1-4 طباعة " العدد الأكبر هو Y والعدد الأصغر هو X "

5. النهاية

6- ارسم خريطة التدفق لبرنامج يحسب مساحة المربع بمعلومية طول الضلع.

المدخلات : طول الضلع

المخرجات : طباعة مساحة المربع

الخطوات

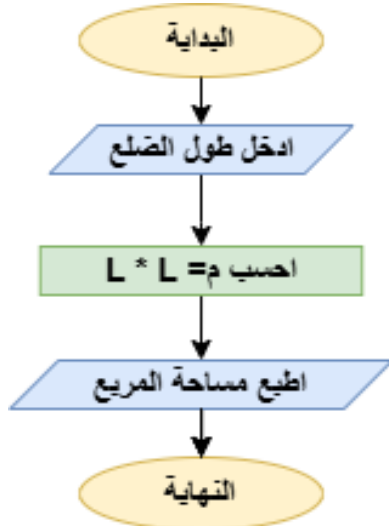
1. البداية

2. إدخال طول الضلع. (L)

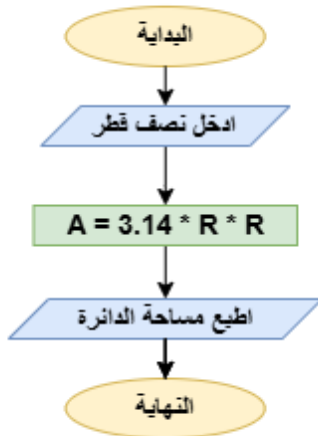
3. حساب مساحة المربع (Area) من خلال المعادلة $Area = L \times L$:

4- طباعة مساحة المربع

5. النهاية



ارسم خريطة التدفق لبرنامج يحسب مساحة الدائرة بمعلومية نصف القطر.



المدخلات : نصف القطر

المخرجات : طباعة مساحة الدائرة

الخطوات

1. البداية

2. إدخال نصف القطر. (R)

3. حساب مساحة الدائرة من خلال المعادلة $3.14 \times R \times R$:

4. طباعة مساحة الدائرة

5. النهاية



احسب الناتج بدون تشغيل الكود أولاً ثم شغله وتأكد من إجابتك مع تحديد أولويه العمليات فيها:

- `print(1+2*3/4.0)`
- `print(20/5*2)`
- `print(10-4+3)`
- `print(50-9*3+2)`
- `print(40-5*4+3)`
- `print(100/5+7-4)`
- `print((50-9)*3+2)`
- `print(70-5*(2+3))`
- `print((60-10)/(5-3)+6)`



قبل ما نحل لازم تبقي عارف

الترتيب: دة

1. الأقواس : () تُنفذ أولاً.
2. الضرب * والقسمة / : في نفس المستوى، وتنفذ من اليسار إلى اليمين.
3. الجمع + والطرح - : في نفس المستوى، وتنفذ من اليسار إلى اليمين.

- `print(1+2*3/4.0) = 2.5`

الخطوات: الأولوية للضرب والقسمة من اليسار لليمين.

- أولاً (الضرب): $2 * 3 = 6$
- ثانياً (القسمة): $6 / 4.0 = 1.5$
- ثالثاً (الجمع): $1 + 1.5 = 2.5$

الناتج النهائي: 2.5 :

- `print(20/5*2)= 8.0`

الخطوات :القسمة والضرب في نفس المستوى، نبدأ من اليسار لليمين.

- أولاً (القسمة) $20 / 5 = 4.0$ بايثون يحول ناتج القسمة لعدد عشري
- ثانياً (الضرب) $4.0 * 2 = 8.0$:

الناتج النهائي :8.0

- `print(10-4+3)= 9`

الخطوات :الطرح والجمع في نفس المستوى، نبدأ من اليسار لليمين.

- أولاً (الطرح) $10 - 4 = 6$
- ثانياً (الجمع) $6 + 3 = 9$:

الناتج النهائي : 9

- `print(50-9*3+2)=25`

الخطوات :الأولوية للضرب

- أولاً (الضرب) $9 * 3 = 27$:
- ثانياً (الطرح من اليسار) $50 - 27 = 23$:
- ثالثاً (الجمع) $23 + 2 = 25$:

الناتج النهائي : 25

- `print(40-5*4+3)=23`

الخطوات :الأولوية للضرب

- أولاً (الضرب) $5 * 4 = 20$:
- ثانياً (الطرح من اليسار) $40 - 20 = 20$:
- ثالثاً (الجمع) $20 + 3 = 23$:

الناتج النهائي :23

- `print(100/5+7-4)=23.0`

الخطوات: القسمة والضرب في نفس المستوى، نبدأ من اليسار لليمين.

- أولاً (القسمة) $100 / 5 = 20.0$:
- ثانياً (الجمع من اليسار) $20.0 + 7 = 27.0$:
- ثالثاً (الطرح) $27.0 - 4 = 23.0$:

الناتج النهائي: 23.0

- `print((50-9)*3+2)= 125`

الخطوات: الأولوية القصوى للأقواس.

- أولاً (فك الأقواس) $50 - 9 = 41$:
- ثانياً (الضرب) $41 * 3 = 123$:
- ثالثاً (الجمع) $123 + 2 = 125$:

الناتج النهائي : 125

- `print(70-5*(2+3))=45`

الخطوات: الأولوية القصوى للأقواس.

- أولاً (فك الأقواس) $2 + 3 = 5$: المعادلة تصبح: $5 * 5 - 70$
- ثانياً (الضرب) $5 * 5 = 25$: المعادلة تصبح: $25 - 70$
- ثالثاً (الطرح) $70 - 25 = 45$:

الناتج النهائي : 45

- `print((60-10)/(5-3)+6)=31.0`

الخطوات: نفك جميع الأقواس أولاً.

- أولاً (القوس الأول) $60 - 10 = 50$:
- ثانياً (القوس الثاني) $5 - 3 = 2$: المعادلة تصبح: $6 + 2 / 50$
- ثالثاً (القسمة) $50 / 2 = 25.0$:
- رابعاً (الجمع) $25.0 + 6 = 31.0$:

الناتج النهائي : 31.0

- `x = 100`
- `salary = 350.25`
- `age = 30`
- `name = "Ahmed"`
- `is_student = False`
- `y = 3.14`
- `is_active = True`

قبل ما نحل نعرف الأول أنوع المتغيرات

- `int (Integer)` : رقم صحيح (مفیش فيه كسور أو علامة عشرية).
- `float` : رقم عشري (بیحتوي على نقطة أو علامة عشرية).
- `str (String)` : نص (أي كلام أو حروف مكتوبة بین علامتي تنصيص " " أو ' ').
- `bool (Boolean)` : قيمة منطقية (بتأخذ قيمة من اثنين بس: یا إما `True` صح، یا إما `False` خطأ)

الحل

- `x = 100` → `int`
- `salary = 350.25` → `float`
- `age = 30` → `int`
- `name = "Ahmed"` → `string`
- `is_student = False` → `bool`
- `y = 3.14` → `float`
- `is_active = True` → `bool`

الشكل الأول: ما ناتج تنفيذ الكود الآتي: السؤال دة بيجيلي الكود وبشوف الكود

هيطلع اي

<<<<<-----السؤال الاول----->>>>>

```
temp = 30
if temp > 35:
    print("الطقس حار جدا")
elif temp >= 25:
    print("الطقس دافئ")
else:
    print("الطقس بارد")
```

شرح الحل مش بيتكتب ف الامتحان

لأكواد الشروط (if / elif / else):

بنشوف قيمة المتغير ونسأل: هل الشرط الأول صح؟
لو صح بننفذه، لو غلط بنروح للشرط اللي بعده."

الإجابة : الطقس دافئ

ليه؟ لأن الحرارة هنا (30).. فلو سألنا هل الـ 30 أكبر من 35؟ لا، فهنسيب أول شرط ونروح للتاني (elif): هل الـ 30 أكبر من أو تساوي 25؟ أيوة

الحل

الناتج:

الطقس دافئ

<<<<<-----السؤال الثاني----->>>>>

```
x = 1
if x == 1:
    print("صح")
```

شرح الحل مش بيتكتب ف الامتحان

هنا بيقول ان الـ x بـ 1

بعدين if لو الـ x == اثنين يساوي يعني
يساوي لو كتبنا = واحدة كدة بقولة ان الـ x بـ 1

ف بيقول لو الـ x بتساوي 1 اطبع كلمة (صح)
Else يعني لو شرط اللي فوق متحققش يعني
لو الـ x مش بتساوي 1 هيطلع (خطأ)

الحل

صح

<<<<-----السؤال الثالث----->>>>

age = 12

score = 55

if (age >= 10 and score >= 50):

print("مؤهل للمشاركة")

else:

شرح الحل مش بيتكتب ف الامتحان

ههنا بيقول ان العمر بيساوي 12 والاسكور بيساوي 55

ف بيقول لو العمر اكبر من او يساوي 10 و السكور اكبر من او يساوي 50 هتطبع مؤهل للمشاركة

غير كده اطبع غير مؤهل

يعني لازم الشرطين يتحققوا عشان يطبع مؤهل

الحل

النتاج:

مؤهل للمشاركة

<<<<-----السؤال الرابع----->>>>

برنامج للتحقق من دخول حديقة

has_ticket = True

is_vip = False

if (has_ticket or is_vip):

print("مسموح بالدخول")

else:

شرح الحل مش بيتكتب ف الامتحان

هنا إحنا بنستخدم (or) يعني "أو"، وده معناه إن شرط واحد بس كفاية يكون صح

هو هنا بيقول ان الشخص معاه تذكره ومش VIP

IF فلو الشخص معاه تذكره او هو VIP اسمح لي بالدخول فالشخص هنا معاه تذكره بس مش VIP فهنا هيسمح بالدخول لان OR مش لازم الاثنين يتحققوا واحده بس ممكن تتحقق والثانيه لا

الحل

النتاج:

مسموح بالدخول

<<<<<-----السؤال الخامس----->>>>>

x = -3

if x > 0:

print("الرقم موجب")

else:

print("الرقم سالب أو صفر")

شرح الحل لمش بيتكتب ف الامتحان

هنا يقول ان X بتساوي -3

وبعد كده الشرط IF لو X اكبر من 0 هتطبع رقم موجب

ELSE غير كده يعني لو الصفر مش اكبر من X اللي هي -3 هيطبع رقم سالب أو صفر

عندي هنا ال X اصغر من 0 هيطبع الرقم سالب

الحل

النتج:

الرقم سالب أو صفر

<<<<<-----السؤال السادس----->>>>>

score = 82

if score >= 90:

print("امتياز")

elif score >= 75:

print("جيد جداً")

elif score >= 60:

print("جيد")

elif score >= 50:

print("مقبول")

شرح الحل لمش بيتكتب ف الامتحان

في أوامر (if و elif) الكمبيوتر بيختبر الشروط بالترتيب من فوق لتحت. الدرجة هنا 82، هيسأل نفسه في أول شرط: هل الـ 82 أكبر من أو تساوي 90؟ لا. هينزل للشرط اللي بعده: هل هي أكبر من أو تساوي 75؟ أبوة صح! فهيطبع "جيد جداً" ويقف على كده ويتجاهل باقي الشروط اللي تحت.

الحل

النتج:

جيد جداً

<<<<-----السؤال السابع----->>>>

```
temp = 50
```

```
if temp < 10 or temp > 30:
```

```
    print("الطقس غير مريح")
```

```
else:
```

```
    print("الطقس معتدل")
```

شرح الحل مش بيتكتب ف الامتحان

هنا إحنا بنستخدم (or) يعني "أو"، وده معناه إن لو شرط واحد بس من الاتنين طلّع صح، هننفذ أمر الطباعة الأولاني. درجة الحرارة هنا 50، فبنسأل: هل الـ 50 أقل من 10؟ لأ.. طب هل الـ 50 أكبر من 30؟ أيوة صح! بما إن فيه شرط اتحقق، يبقى هيطبع "الطقس غير مريح".

الحل

النتائج:

الطقس غير مريح

<<<<-----السؤال الثامن----->>>>

```
for i in range(3):
```

```
    print("اشرب كوب ماء")
```

شرح الحل مش بيتكتب ف الامتحان

لأكواد التكرار (for loop):

الشرح: "حلقة (for) بتخلي الكمبيوتر يلف ويكرر أمر الطباعة عدد معين من المرات ورا بعض."

أمر range(3) معناه ببساطة كرر الخطوة دي 3 مرات فكدّه إحنا بنأمر الكمبيوتر إنه يطبع جملة "اشرب كوب ماء" ثلاث مرات ورا بعض كل مرة في سطر لوحدها.

الحل

النتائج:

اشرب كوب ماء

اشرب كوب ماء

اشرب كوب ماء

<<<<<-----السؤال التاسع----->>>>>

```
for i in range(1, 6):
```

```
    print("قم بتمرين الضغط رقم", i)
```

```
print("تم الانتهاء من التمرين اليوم")
```

شرح الحل مش بيتكتب ف الامتحان

لأكواد التكرار (for loop):

أمر range(1, 6) معناه "ابدأ العد من رقم 1 لحد قبل رقم 6" (يعني من 1 لـ 5). فالكومبيوتر هيلف 5 مرات ويطبع الجملة ويغير الرقم (i) في كل لفة. ولما اللف يخلص خالص، هيطبع جملة "تم الانتهاء" مرة واحدة بس في الآخر عشان هي مكتوبة بره حلقة الـ (for).

الحل

النتاج:

قم بتمرين الضغط رقم 1
قم بتمرين الضغط رقم 2
قم بتمرين الضغط رقم 3
قم بتمرين الضغط رقم 4
قم بتمرين الضغط رقم 5
تم الانتهاء من التمرين اليوم

<<<<<-----السؤال العاشر----->>>>>

```
for i in range(1, 5):
```

```
    if i == 2:
```

```
        print("الطالب رقم 2 غائب، تم تخطي تسجيله")
```

```
        continue # تخطي الطالب الغائب
```

```
    print("تم تسجيل حضور الطالب رقم", i)
```

شرح الحل مش بيتكتب ف الامتحان

لأكواد التكرار (for loop):

اللوب هيعد من 1 لـ 4 (لأن range(1, 5) بتقف قبل الـ 5). لما يوصل لرقم 2، هيطبع رسالة الغياب، وهنا بييجي دور أمر continue اللي بيقول للكومبيوتر "نط اللفة دي وتجاهل السطور اللي تحتها وكمل من الرقم اللي بعده"، عشان كده متسجلش حضور لرقم 2 وكمل للرقم 3 و 4 عادي.

الحل

النتاج:

تم تسجيل حضور الطالب رقم 1
الطالب رقم 2 غائب، تم تخطي تسجيله
تم تسجيل حضور الطالب رقم 3
تم تسجيل حضور الطالب رقم 4

<<<-----السؤال الحادي عشر----->>>>

```
password = "1234"
for i in range(3):
    user = input("ادخل الرقم السري:")
    if user == password:
        print("تم فتح اللابتوب")
```

شرح الحل مش بيتكتب ف الامتحان

لأنكوات التكرار (for loop):

أمر range(3) بيدي للمستخدم 3 محاولات عشان يكتب الباسورد. لو كتبه صح، هيطبع رسالة الفتح، وهنا بييجي دور أمر break اللي بيوقف التكرار (اللوب) فوراً ويبقول للكمبيوتر "خلاص وقف سؤال"، عشان هو دخل الباسورد صح ومش محتاجين نخليه يكمل المحاولات الباقية

الحل

الناتج: (لو المستخدم كتب 1234):

ادخل الرقم السري: 1234
تم فتح اللابتوب

<<<<-----السؤال الثاني عشر----->>>>

```
city = "cairo"
street = "Maddi"
address = city + "," + street
print("العنوان الكامل هو:", address)
```

شرح الحل مش بيتكتب ف الامتحان

النصوص والقوائم (Lists):

هنا إحنا بنعمل حاجة اسمها "دمج النصوص". بنستخدم علامة الزائد + عشان نلحق الكلمات في بعضها. حطينا اسم المدينة (cairo)، ولزقنا فيها فاصلة (,)، ولزقنا فيها اسم الشارع (Maddi)، وجمعناهم كلهم في متغير واحد اسمه address وطبعناه في الآخر.

الحل

الناتج:

العنوان الكامل هو: cairo,Maddi

<<<<-----السؤال الثالث عشر----->>>>

```
word = "Python"
for i in range(0, len(word), 2):
    print("النتيجة النهائية هي", word[i])
```

شرح الحل مش بيتكتب ف الامتحان

النصوص والقوائم (Lists)

اللوبي هنا بيلف على حروف كلمة "Python". بس أمر `range(0, len(word), 2)` معناه "ابدأ من الأول، بس نط خطوتين كل مرة" (يعني الحرف رقم 0، وبعدين 2، وبعدين 4)، عشان كده طبع الحرف الأول والثالث والخامس بس.

الحل

الناتج:

النتيجة النهائية هي P

النتيجة النهائية هي t

النتيجة النهائية هي o

<<<<-----السؤال الرابع عشر----->>>>

```
tasks = [ ]
for i in range(3):
    task = input("أدخل مهمة اليوم : ")
    tasks.append(task)

print("قائمة المهام :")
```

شرح الحل مش بيتكتب ف الامتحان

النصوص والقوائم (Lists)

عملنا قائمة فاضية []. اللوب طلب من المستخدم يدخل 3 مهام، واستخدمنا الدالة `append()` عشان نضيف كل مهمة جوه القائمة. وفي الآخر عملنا لوب ثاني عشان نطبعهم تحت بعض وجنبهم شرطة (-).

الحل

الناتج: (لو افترضنا المستخدم كتب: مذاكرة، لعب، نوم):

قائمة المهام :

-مذاكرة

-لعب

-نوم

<<<<<-----السؤال الخامس عشر----->>>>>

```
orders = [ ]

for i in range(3):
    new = input("ادخل 3 الطلبات: ")
    orders.append(new)

last_order = orders.pop()

print("تم توصيل الطلب", last_order)
```

شرح الحل مش بيتكتب ف الامتحان

النصوص والقوائم (Lists):

بعد ما ضفنا 3 طلبات للقائمة، استخدمنا الدالة `pop()` والتي وظيفتها "حذف آخر عنصر تم إضافته للقائمة واسترجاعه"، فمسحت (كولا) من القائمة وطبعتها لوحدها، وبعدين طبعت القائمة باللي فاضل فيها.

الحل

الناتج: (لو افترضنا الطلبات: بيتزا، برجر، كولا):

تم توصيل الطلب كولا

الطلبات المتبقية ['بيتزا', 'برجر']

<<<<<-----السؤال السادس عشر----->>>>>

```
A = {1, 2, 3}
B = {3, 4, 5}

A.add(6)
A.remove(2)

print(A | B) # union
print(A & B) # intersection
```

شرح الحل مش بيتكتب ف الامتحان

المجموعات (Sets):

في الأول ضفنا رقم 6 للمجموعة A ومسحنا منها رقم 2 (فبقت A فيها 1, 3, 6).

السطر الأول بيطلع الاندماج (|) يعني كل الأرقام بدون تكرار. السطر الثاني بيطلع التقاطع (&) يعني المشترك بس. السطر الثالث بيطلع الفرق (-) يعني اللي موجود في A ومش موجود في B.

الحل

الناتج:

{1, 3, 4, 5, 6}

{3}

{1, 6}

<<<<<-----السؤال السابع عشر----->>>>>

```
data = {
    1: 'Admin',
    2: 'Editor',
    3: 'Reader'
}
```

```
print(data[2])
```

شرح الحل مش بيتكتب ف الامتحان

القواميس: (Dictionaries)

القاموس بيتكون من مفتاح وقيمة (Key: Value). هنا طلبنا نطبع القيمة الخاصة بالمفتاح رقم 2، وبعتها القيمة الخاصة بالمفتاح رقم 1 باستخدام دالة get()، فطبع الوظائف بتاعتهم.

الحل

الناتج:

Editor
Admin

<<<<<-----السؤال الثامن عشر----->>>>>

```
A = {"a": 1, "b": 2, "c": 3}
```

```
B = {"b": 20, "d": 4}
```

```
A["e"] = 5
```

```
B["a"] = 10
```

```
del A["b"]
```

```
B.pop("d")
```

شرح الحل مش بيتكتب ف الامتحان

القواميس: (Dictionaries)

عندنا قاموسين A و B. في الأول ضفنا عنصر جديد للقاموس A وهو ("e": 5)، وضمنا عنصر للقاموس B وهو ("a": 10). بعدين استخدمنا أمر del عشان نمسح العنصر "b" من القاموس A، واستخدمنا دالة pop() اللي برضه وظيفتها تمسح العنصر "d" من القاموس B. ولما طبعناهم في الآخر، ظهوروا بالشكل الجديد بعد التعديل (الإضافة والحذف).

الحل

الناتج:

```
{'a': 1, 'c': 3, 'e': 5}
{'b': 20, 'a': 10}
```

<<<<<-----السؤال التاسع عشر----->>>>>

```
def multiply(a, b):
```

```
    return a * b
```

```
def add(a, b):
```

```
    return a + b
```

```
result1 = multiply(3, 4)
```

```
result2 = add(5, 2)
```

شرح الحل مش بيتكتب ف الامتحان

الدوال: (Functions)

هنا إحنا عملنا دالتين (يعني كودين جاهزين للاستخدام):
الأولى اسمها multiply وظيفتها تضرب أي رقمين في بعض، والثانية اسمها add وظيفتها تجمع رقمين.

لما نادينا على دالة الضرب وبعثناها الأرقام (3 و 4) حسبناها وخزنت الناتج (12)، ولما نادينا على دالة الجمع وبعثناها (5 و 2) حسبناها وخزنت الناتج (7)، وفي الآخر استخدمنا أمر الطباعة عشان نعرض النتيجة على الشاشة.

الحل

الناتج:

12

7

<<<<<-----السؤال العشرين----->>>>>

```
student = {
```

```
    "name": "Ali",
```

```
    "age": 21,
```

```
    "grade": "B"
```

```
}
```

```
student["age"] = 22
```

```
student["city"] = "Cairo"
```

```
del student["grade"]
```

شرح الحل مش بيتكتب ف الامتحان

الدوال: (Functions)

إحنا هنا عملنا قاموس (Dictionary) بيخزن بيانات طالب (اسمه، عمره، وتقديره). في السطور اللي بعدها عملنا 3 تعديلات:

غيرنا عمره لـ 22.

ضفنا معلومة جديدة وهي المدينة (Cairo).

استخدمنا أمر del عشان نمسح التقدير (grade) من القاموس خالص.

ولما طبعناه في الآخر، ظهر القاموس بشكله الجديد بعد التحديث.

الحل

الناتج:

```
{'name': 'Ali', 'age': 22, 'city': 'Cairo'}
```

```
def multiply(a, b):
```

```
    return a * b
```

```
def add(a, b):
```

```
    return a + b
```

```
result1 = multiply(3, 4)
```

```
result2 = add(5, 2)
```

```
print(result1)
```

```
print(result2)
```

شرح الحل مش بيتكتب ف الامتحان

دوال: (Lambda)

عملنا دالة اسمها multiply بتضرب رقمين، ودالة اسمها add بتجمعهم، وبعثناهم الأرقام وطبعنا الناتج.

أما السطر الأخير فده دالة سريعة من سطر واحد اسمها lambda، وظيفتها هنا تأخذ الرقم وتضربه في نفسه، فلما بعثناها رقم 3، ضربت (3 × 3) وطبعت.

الحل

الناتج:

12

7

9

<<<<<-----السؤال الاول----->>>>>

1. اكتب برنامج بايثون يأخذ عمر المستخدم ويقرر إذا كان مسموح له بالدخول أم لا:

- ✚ اطلب من المستخدم إدخال عمره باستخدام input.
- ✚ إذا كان العمر أكبر من أو يساوي 18، اطبع: "مسموح بالدخول".
- ✚ إذا كان أقل من 18 اطبع: "غير مسموح بالدخول".

الحل

```
age = int(input("أدخل عمرك: "))
if age >= 18:
    print("مسموح بالدخول")
else:
    print("غير مسموح بالدخول")
```

شرح الحل مش بيتكتب ف الامتحان

استخدمنا input عشان نطلب من المستخدم يكتب عمره، و int عشان نحول الكلام لرقم. بعيين سألنا بـ if: لو العمر 18 أو أكبر هنطبع "مسموح"، ولو أصغر هنروح لـ else ونطبع "غير مسموح".

<<<<<-----السؤال الثاني----->>>>>

2. اكتب برنامج بايثون يسأل المستخدم عن عدد النقاط التي حصل عليها في لعبة ويقرر إذا فاز أو لا:

- ✚ اطلب من المستخدم إدخال عدد النقاط باستخدام input.
- ✚ إذا كانت النقاط أكبر من أو تساوي 50، اطبع: "مبروك! لقد فزت".
- ✚ إذا كانت أقل من 50 اطبع: "حاول مرة أخرى".

الحل

```
points = int(input("أدخل عدد النقاط: "))
if points >= 50:
    print("مبروك! لقد فزت")
else:
    print("حاول مرة أخرى")
```

شرح الحل مش بيتكتب ف الامتحان

نفس فكرة البرنامج اللي قبله، بناخد النقط من المستخدم، وبنسأل: لو النقط 50 أو أكثر بيطلع رسالة الفوز، ولو أقل بيطلع رسالة المحاولة مرة ثانية.

<<<<-----السؤال الثالث----->>>>

3. اكتب برنامج بايثون يسأل المستخدم عن عدد ساعات نومه ويقرر إذا كان كافيًا أو لا:

⚡ اطلب من المستخدم إدخال عدد ساعات النوم باستخدام `input`.

⚡ إذا كانت الساعات أكبر من أو تساوي 7، اطبع: "نومك كافي".

⚡ إذا كانت أقل من 7 اطبع: "تحتاج للمزيد من النوم".

الحل

```
sleep_hours = int(input("أدخل عدد ساعات نومك: "))
```

```
if sleep_hours >= 7:
```

```
    print("نومك كافي")
```

```
else:
```

```
    print("تحتاج للمزيد من النوم")
```

شرح الحل مش بيتكتب ف الامتحان

طلبنا عدد ساعات النوم، وبختر الشرط: لو نام 7 ساعات أو أكثر بنطبع إن نومه كافي، ولو الرقم أصغر من 7 بننصح بنام أكثر.

<<<<-----السؤال الرابع----->>>>

4. اكتب برنامج يقرر إذا كان الرقم مناسب للعبة معينة:

⚡ الرقم يجب أن يكون أكبر من أو يساوي 10 و أقل من أو يساوي 20.

⚡ إذا تحقق الشرطان اطبع "الرقم مناسب للعبة".

⚡ إذا لم يتحقق أي شرط، اطبع "الرقم غير مناسب".

الحل

```
num = int(input("أدخل الرقم: "))
```

```
if num >= 10 and num <= 20:
```

```
    print("الرقم مناسب للعبة")
```

```
else:
```

شرح الحل مش بيتكتب ف الامتحان

استخدمنا هنا معامل `and` (و)، وده معناه إن الشرطين لازم يتحققوا مع بعض.. يعني لازم الرقم يكون 10 أو أكبر، وكمان يكون 20 أو أصغر، عشان يطبع إنه مناسب.

<<<<<-----السؤال الخامس----->>>>>

5. اكتب برنامج يطلب من المستخدم إدخال رقمين ويطبع أكبرهما، أو يطبع أنهما متساويان إذا كانا كذلك.

الحل

```
num1 = int(input("أدخل الرقم الأول: "))
num2 = int(input("أدخل الرقم الثاني: "))

if num1 > num2:
    print("الرقم الأكبر هو:", num1)
elif num2 > num1:
    print("الرقم الأكبر هو:", num2)
```

شرح الحل مش بيتكتب ف الامتحان

طلبنا رقمين، لو الأول أكبر هيطبعه، ولو الثاني أكبر هيطبعه (elif)، أما لو الشرطين دول طلعا غلط، يبقى أكيد هما متساويين فهيطبع الجملة الأخيرة في (else).

<<<<<-----السؤال السادس----->>>>>

6. اكتب برنامج يطلب من المستخدم إدخال عمره ويحدد إذا كان طفل أو بالغاً ان كان أكبر من 18 سنة.

الحل

```
age = int(input("أدخل عمرك: "))

if age > 18:
    print("أنت بالغ")
else:
```

شرح الحل مش بيتكتب ف الامتحان

الشرح: شرط بسيط جداً، بنسأل لو العمر أكبر من 18 بيطبع "بالغ"، وأي رقم تاني (18 أو أصغر) بيطبع "طفل".

<<<<-----السؤال السابع----->>>>

7. مثال : اكتب برنامج يقوم بما يلي:

استخدم حلقة for لتسجيل حضور 5 طلاب.

في كل مرة اطبع "تم تسجيل حضور الطالب رقم."

بعد انتهاء الحلقة، اطبع رسالة نهائية: "اكتمل تسجيل الحضور."

الحل

```
for i in range(1, 6):
    print("تم تسجيل حضور الطالب رقم", i)

print("اكتمل تسجيل الحضور")
```

شرح الحل مش بيتكتب ف الامتحان

عملنا لوب (for) يعد من 1 لـ 5، ويطبع جملة الحضور وجنبها رقم الطالب. ولما يخلص لف، هيطبع الجملة الختامية مرة واحدة بره اللوب.

<<<<-----السؤال الثامن----->>>>

8. اكتب برنامج تتأكد فيه ان المستخدم ادخل رقم الهاتف 10 ارقام.

الحل

```
phone = input("أدخل رقم الهاتف: ")

if len(phone) == 10:
    print("رقم الهاتف صحيح")
else:
```

شرح الحل مش بيتكتب ف الامتحان

استخدمنا دالة len() اللي بتعد الحروف أو الأرقام. فبنسأل: هل طول الرقم اللي المستخدم كتبه بيساوي 10 بالظبط؟ لو أيوة بيطلع صحيح، لو لا بيطلع إنه غير صحيح.

<<<<-----السؤال التاسع----->>>>

9. اكتب برنامجاً بلغة بايثون يقوم بالآتي:

✚ إنشاء قائمة باسم subjects تحتوي على المواد: "Math", "English", "Arabic" :

✚ استخدام حلقة تكرارية for loop للمرور على كل مادة في القائمة.

✚ طباعة اسم المادة وبجانبها جملة.

الحل

```
subjects = ["Math", "English", "Arabic"]
```

```
for subject in subjects:
```

```
    print(subject, "مادة ممتعة")
```

شرح الحل مش بيتكتب ف الامتحان

عملنا قائمة فيها 3 مواد، واستخدمنا (for loop) عشان يمر على مادة مادة جوه القائمة ويطبع اسمها وجنبه كلمة (مادة ممتعة).

<<<<-----السؤال العاشر----->>>>

10. اكتب دالة تستقبل عدداً صحيحاً n وتقوم بإرجاع مربع هذا العدد.

الحل

```
def square(n):
```

```
    return n * n
```

شرح الحل مش بيتكتب ف الامتحان

المربع يعني نضرب الرقم في نفسه. الدالة دي بتأخذ الرقم n وتضربه في n وترجع الناتج باستخدام return.

<<<<<-----السؤال الحادي عشر----->>>>>

11. اكتب برنامج لعمل الدالة التالية: دالة تقوم بالتحقق إذا كان الرقم زوجي أم فردي.

الحل

```
def check_even_odd(num):
    if num % 2 == 0:
        print("الرقم زوجي")
    else:
```

شرح الحل مش بيتكتب ف الامتحان

أي رقم يقبل القسمة على 2 بدون باقي
(num % 2 == 0) يكون زوجي، ولو
اتبقى حاجة يكون فردي.

<<<<<-----السؤال الثاني عشر----->>>>>

12. اكتب الدالة باسم calculate_bill تستقبل متغيرين:

- price السعر
- tax الضريبة
- تقوم الدالة بحساب الإجمالي عن طريق المعادلة: الإجمالي = السعر + (السعر × الضريبة).

الحل

```
def calculate_bill(price, tax):
    total = price + (price * tax)
    return total
```

شرح الحل مش بيتكتب ف الامتحان

الدالة دي بتأخذ السعر والضريبة،
وبتحسب المعادلة المطلوبة، وبعدين
بترجع الإجمالي total اللي بيستدعيها.

<<<<<-----السؤال الثالث عشر----->>>>>

13. اكتب دالة تستقبل عددين صحيحين وتقوم بارجاع ناتج القسمة، مع مراعاة معالجة حالة القسمة على صفر.

الحل

```
def divide(num1, num2):  
    if num2 == 0:  
        return "خطأ: لا يمكن القسمة على صفر"  
    else:  
        return num1 / num2
```

شرح الحل مش بيتكتب ف الامتحان

في الرياضة ممنوع نقسم على صفر!
فالدالة بتسأل الأول: هل الرقم الثاني
num2 بصفر؟ لو أيوة بتطبع
رسالة خطأ، لو لا بتقسم عادي
وترجع الناتج.

<<<<<-----السؤال الرابع عشر----->>>>>

14. اكتب دالة تستقبل عددًا صحيحًا وتقوم بطباعة "Hello" بعدد مرات هذا الرقم.

الحل

```
def print_hello(n):  
    for i in range(n):  
        print("Hello")
```

شرح الحل مش بيتكتب ف الامتحان

الدالة بتأخذ رقم n، وبتعمل حلقة تكرار (for loop)
تلف عدد المرات دي وتطبع كلمة Hello في كل لفه.

<<<<<-----السؤال الخامس عشر----->>>>>

15. اكتب دالة تستقبل عددًا وتقوم بإرجاع نصفه (قسمة على 2).

الحل

```
def half(num):
    return num / 2
```

شرح الحل مش بيتكتب ف الامتحان

دالة بسيطة جداً، بتأخذ الرقم وتقسّمه على 2 عشان تجيب نصه وترجع الناتج.

<<<<<-----السؤال السادس عشر----->>>>>

16. اكتب دالة تستقبل عددين وتقوم بإرجاع Equal إذا كانا متساويين، وإلا.

الحل

```
def check_equal(num1, num2):
    if num1 == num2:
        return "Equal"
    else:
```

شرح الحل مش بيتكتب ف الامتحان

الدالة بتقارن الرقمين باستخدام معامل التساوي ==. لو متساويين هترجع كلمة Equal، وإلا هترجع Not Equal.

صَلِّ عَلَى النَّبِيِّ ﷺ

بالتوفيق